Software Development Life Cycle

Stage 1: Planning

Project Overview

This project is a CPU Scheduling Algorithm Simulator web application that helps students visualize how different scheduling algorithms work. It will allow the user to input any number of processes with their desired arrival time and burst time. The user can then select an algorithm they wish to use such as FCFS, SJF, SRTF, RR, and Priority. The goal is to make an easy way for anyone interested to visualize and play around with different CPU scheduling algorithms so they can strengthen their understanding of the topic.

Problem Statement

Students studying CPU scheduling algorithms struggle to verify whether their Gantt charts they wrote down are actually correct. Students who try writing out Gantt charts for the first time may not realize or understand what they are doing wrong. The resources students are left with to compare their answers too are diagrams from textbooks, class notes, or online videos. These resources help show how to write your gantt chart but it doesn't have the personalization to check if the student's solution is actually correct. With all the different scheduling algorithms the student has to learn, they might get overwhelmed and confused with the different ways of approaching each algorithm.

End Users

The end-user for my project is someone who is studying CPU scheduling algorithms for the first time and needs a reference for how it works. The user wants a fast and simple way to experiment with different algorithms such as FCFS, SJF, SRTF, RR, and Priority.

Another end user is a person who would use the simulator to show others a visual representation of scheduling algorithms.

Project Goals

- ★ Create a tool for simulating CPU scheduling algorithms.
- ★ Make the web application free to use with no unnecessary forced signup.
- ★ Make it interactive and easy to use.
- ★ Nice modern design with a user friendly interface.
- ★ Allow users to input custom processes with arrival times, burst times, and priority.
- ★ Allow users to select an algorithm of their choice.
- ★ Generate a gantt chart with all relevant information the user needs.

Stage 2: Requirements

Functional Requirements

Input

- ★ Users can select a scheduling algorithm from a list that will contain FCFS, SJF, SRTF, RR, and Priority.
- ★ Users can create multiple processes with arrival times, burst times. Priority will be an option for processes if they selected the Priority Scheduling algorithm.
- ★ When both the algorithm and processes are accounted for, the user can submit to run the simulation.

Output

- ★ The user is shown a gantt chart that shows the processes executing.
- ★ The user should also be shown the average waiting time, the turnaround time, and the throughput.
- ★ There should also be a table of the processes with all their data.

Backend

- ★ The backend REST API needs to take and process the data that the user submits.
- ★ The API needs to run the scheduling algorithm that the user selected from the processes that are provided.

Non-functional Requirements

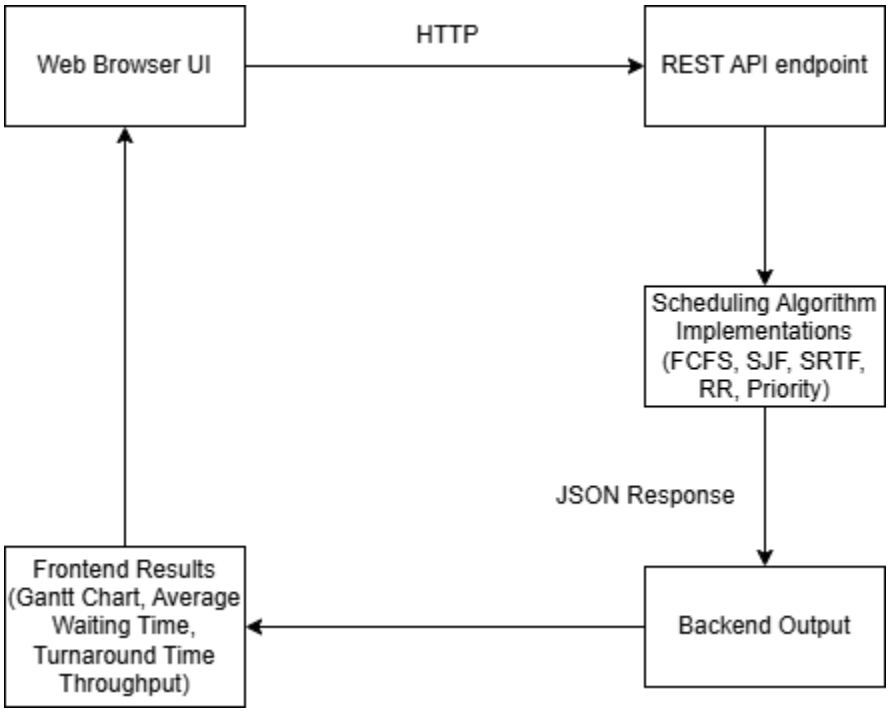
- ★ The user interface has to be simple and easy to use.
- ★ The web application should be able to run on desktop and mobile devices.
- ★ The simulation should run very fast so the user shouldn't be waiting long for the results.
- ★ The algorithms have to be implemented correctly and there should be no mistakes in the output.

Conditions

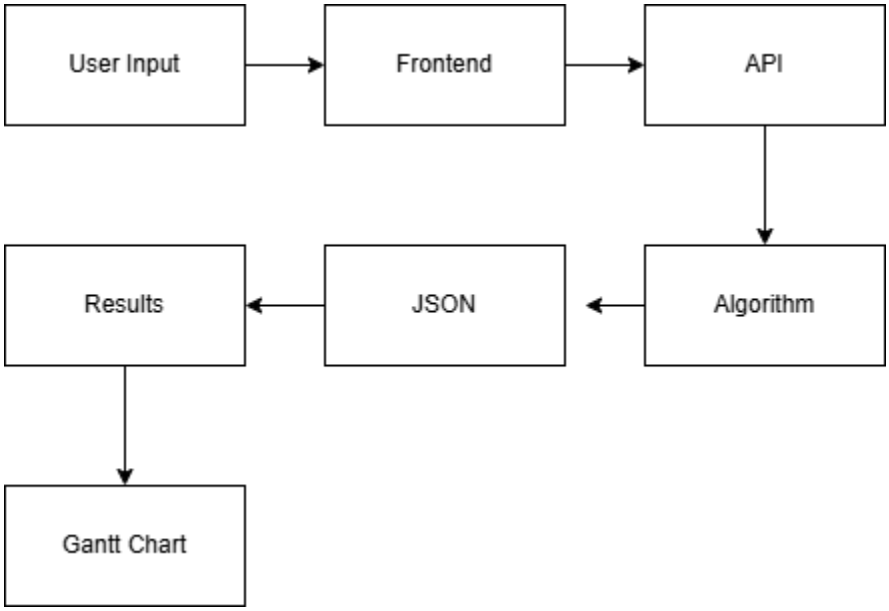
- ★ There will be a separate frontend and backend logic that will use REST API calls to communicate.
- ★ To use the CPU scheduling simulator, the user needs a web browser.
- ★ We do not need a database for this project because no user information needs to be stored.

Stage 3: Design

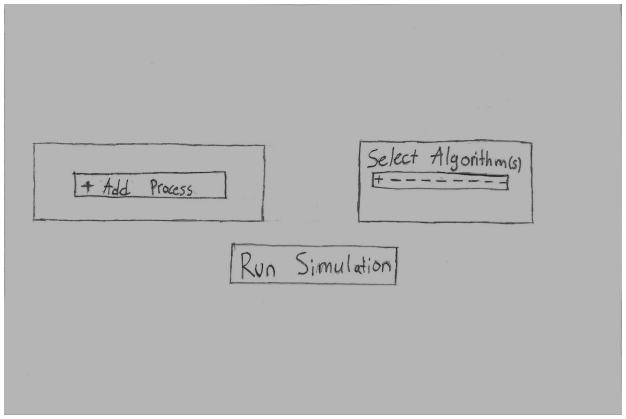
System Architecture

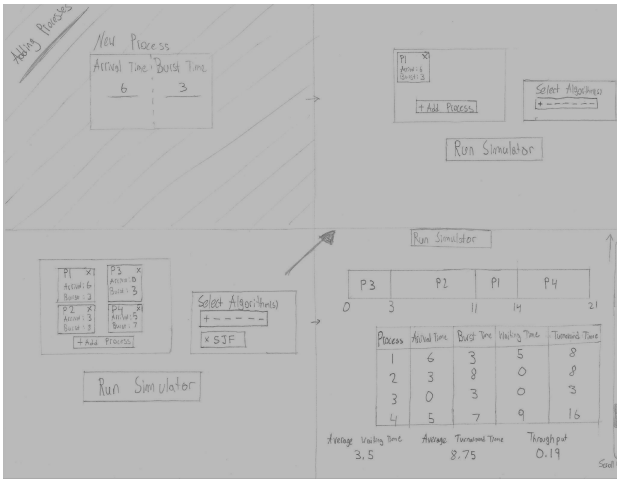


Data Flow Diagram

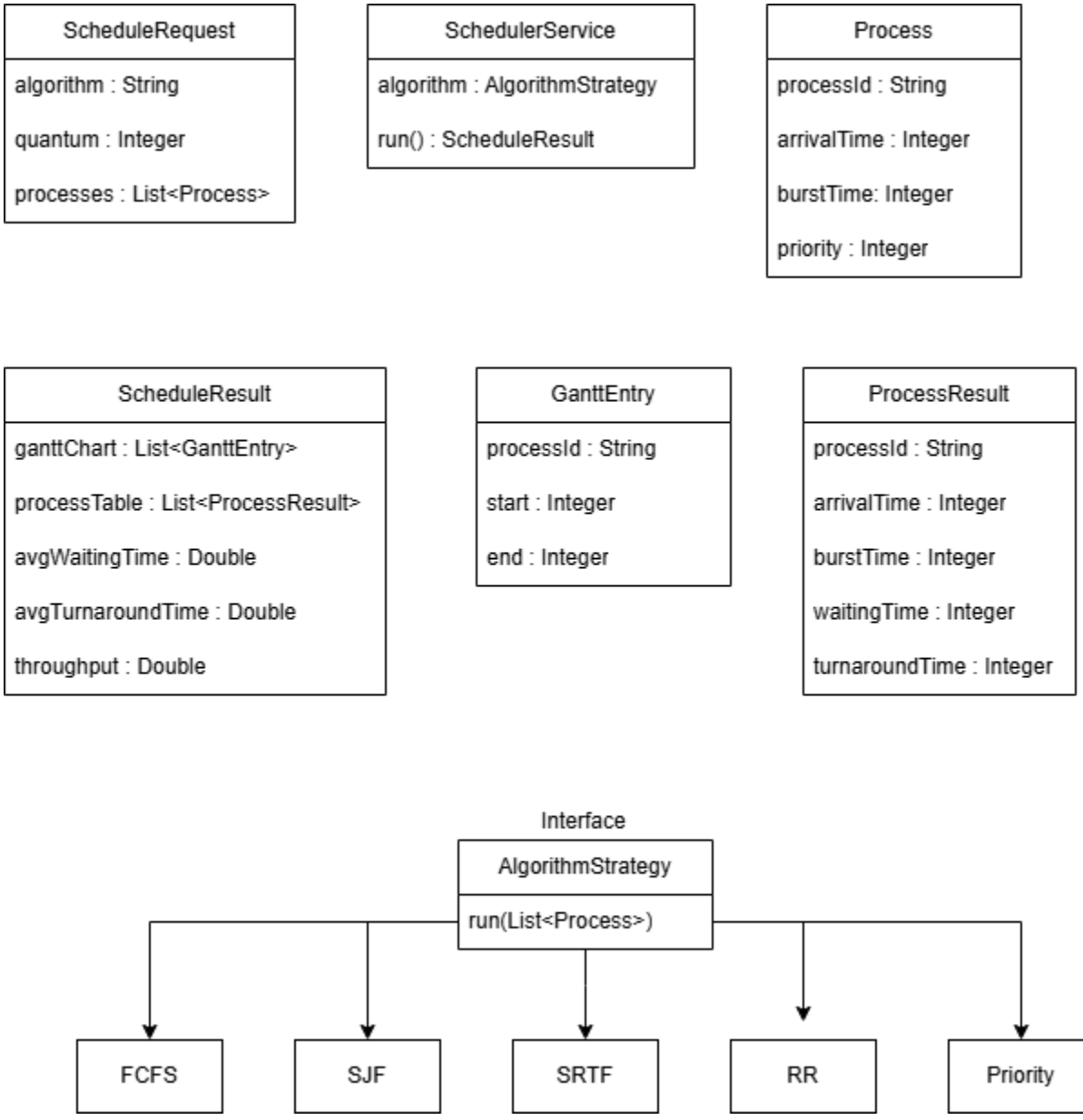
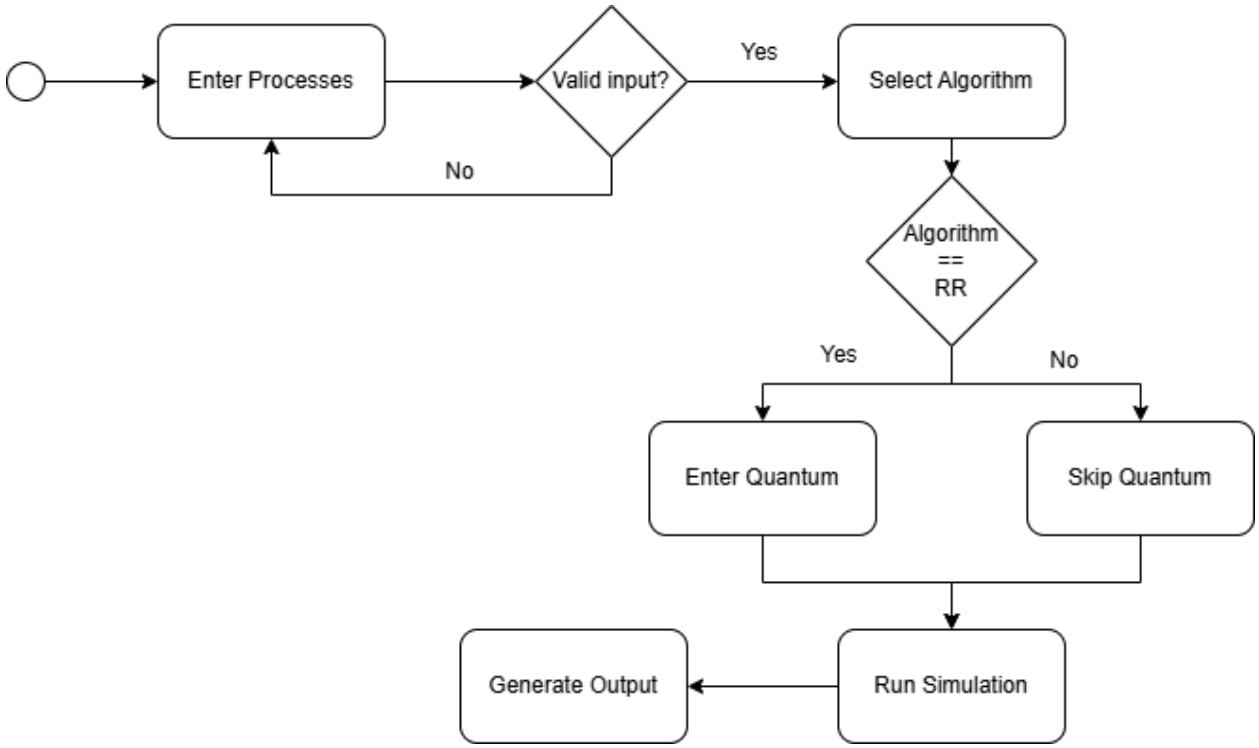


Wire Frame





Class Diagram



Activity Diagram